

SVM and CNN Experiments

SVM

Methodology

The classification task is approached using a custom-built, multi-class Support Vector Machine (SVM) leveraging a One-vs-Rest (OvR) strategy. The pipeline consists of data preprocessing, dimensionality reduction, model training, and evaluation.

Data Preprocessing

The Fashion MNIST dataset is initially loaded and subjected to standard scaling (Z-score normalization). This process centers the data to have a mean of zero and scales it to have a standard deviation of one, which is critical for ensuring that the gradient descent optimization converges efficiently and that features with larger numerical ranges do not dominate the learning process.

Dimensionality Reduction

To improve computational efficiency and reduce noise, Principal Component Analysis (PCA) is implemented from scratch to project the 784-dimensional image vectors into a lower-dimensional space. The PCA implementation:

- Mean-centers the normalized data.
- Computes the covariance matrix ¹of the features.
- Calculates eigenvalues and eigenvectors using custom eigen vec computation or (`np.linalg.eigh`).
- Sorts the eigenvectors in descending order of their eigenvalues to select the top k principal components.

Model Architecture

The classifier is a `MultiClassSVM` consisting of 10 independent binary `SupportVectorModel` instances, one for each class in the dataset.

- **Training Strategy:** During training, the labels are binarized (1 for the target class, -1 for the rest).
- **Optimization:** The model uses Stochastic Gradient Descent (SGD) with a Pegasos decaying learning rate schedule. The learning

¹ The complete code is available here to reproduce https://github.com/pranamyapatil/I_CourseWork.git

rate at iteration t is defined as $\eta_t = \frac{\eta_0}{1 + \eta_0 \alpha t}$, which prevents oscillation near the global minimum.

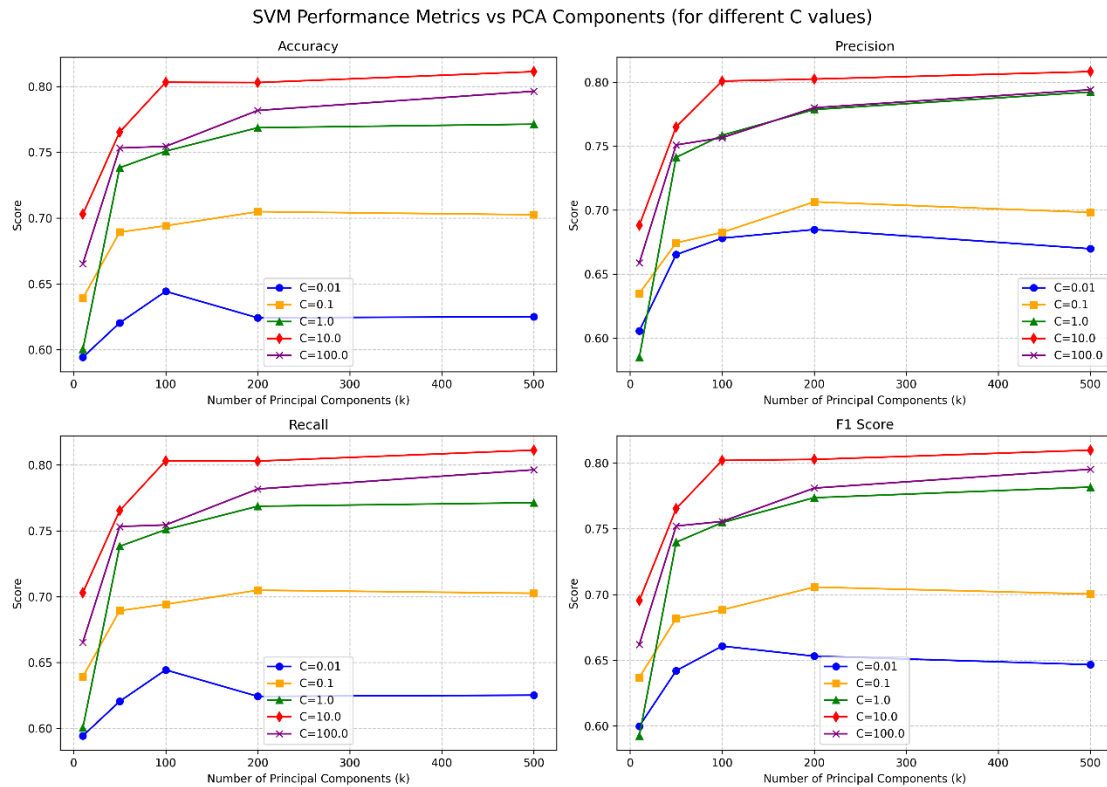
- **Loss Function:** The optimization minimizes the Hinge Loss combined with L2 regularization. The gradient update explicitly checks the margin condition $y_i(w^T x_i + b) < 1$. If violated, the weights are updated using both the hinge loss and regularization terms; otherwise, only the regularization term is applied.

Experimental Setup

The experiments are designed to test the model's performance across different levels of regularization and data compression.

- **Fixed Hyperparameters:** The initial learning rate is set to 0.01, and the number of SGD iterations is fixed at 10,000 per binary classifier.
- **Variable Hyperparameters:**
 - Regularization parameter $C \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$. The L2 penalty coefficient is calculated as $\alpha = 1/C$.
 - Number of PCA components $k \in \{10, 50, 100, 200, 500\}$.

Results and Analysis



As we can see from the diagram we get best results with C around 10.0 and pca number of components k (between 100-200). More than 200 either we don't see huge improvement or it slightly degrades due to noise in the image.

CNN

Methodology

We use couple of layers of cnn followed by ffn (feed forward network) to output logits as number of output class labels.

Data Preprocessing

For CNN based classifier we use similar data processing as SVM based classifier (ie:- Z Transform / normalize the data)

Model Architecture

We use 2 layers of sequential (2D CNN, BatchNorm, ReLU activation followed by pooling).

For the 1st block we use kernel size as (3,3) with padding as (1,) ,num_filters as 16 and maxpooling with kernel size as 2 and stride as 2 (effectively reducing the width and height of input 2d image to half)

For the 2nd block we use kernel size as (3,3) with padding as (1,) ,num_filters as 32 and maxpooling with kernel size as 2 and stride as 2 (effectively reducing the width and height of input 2d image to half)

The output layer is an ffn which projects from output of cnn to 128 to 10 (number of output classes)

Experimental Setup

We use adam optimizer with linear scheduler and cross entropy loss. We train for 5 epochs with batch size as 64 and initial lr as 1e-3. We have trained models on nvidia tesla T4 gpus on google colab.

Results and Analysis

We get following metric scores for fashion_mnist image classification task

Accuracy: 91.55%

Macro Precision: 0.9150

Macro Recall: 0.9155

Macro F1 Score: 0.9150

If we train for more epochs or use denser networks we can get better scores. Also we have incorporated regularization via dropout after cnn and in ffn to avoid overfitting to training data.